File Metadata Storage

facts in a database, bytes in a bucket

Watch the 4-min video

In one line: store file **metadata** (name, folder, owner, permissions, size, hash, versions) in a database and the **content** in object storage, linked by an object key; a content hash catches corruption and, used as the key, deduplicates identical content, while versions become rows that point at immutable bytes.

1 Everything in one place is slow

A file service keeps bytes, names, folders, owners and sharing settings together, say as objects whose keys are paths, with facts in object metadata. Listing a folder means a `LIST` over a prefix; renaming a folder means copying and deleting every object under it; checking whether a user may see a file means fetching its metadata; "files I modified last week" can't be answered at all. Bytes and facts have opposite needs: bytes are big, immutable and read sequentially; facts are tiny, change often and must be queried, indexed and updated atomically.

2 Separate metadata from content

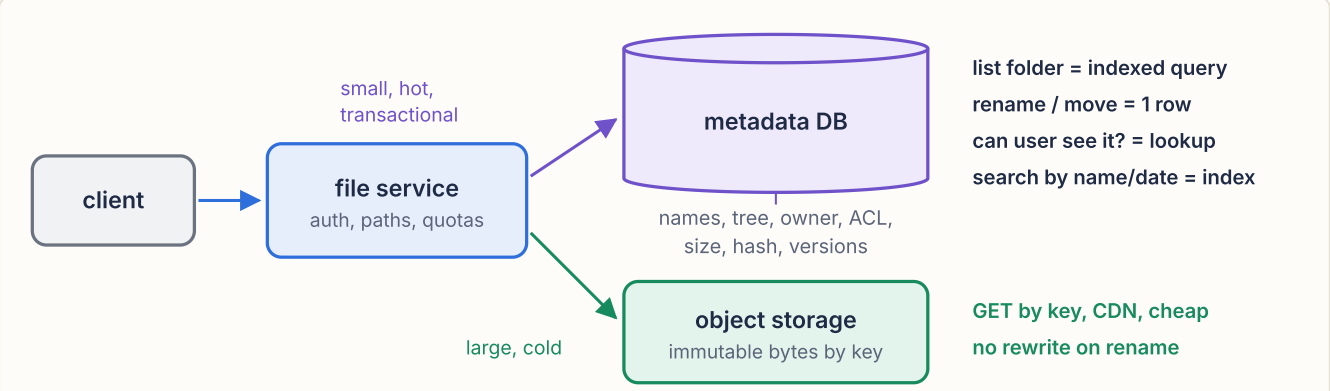


Figure 1. The metadata path is small and transactional; the content path is large and immutable. Each gets the store built for it.

Content goes to object storage under a key the system generates (a UUID or a hash, never the user's path). Metadata goes to a relational or key-value database. Every metadata row points at its content by that key. The split pays off immediately:

- **Listing** a folder is an index range scan on `(parent_id, name)`, with sorting and paging.
- **Rename or move** updates one row; the bytes never move.
- **Permission checks** are database lookups (with inheritance from parent folders), not storage calls.
- **Search** by name, size, type or date uses indexes; a search engine can be fed from the same rows.
- **Transactions**: a move between folders, a quota update and an audit entry commit together.

3 Modelling the tree

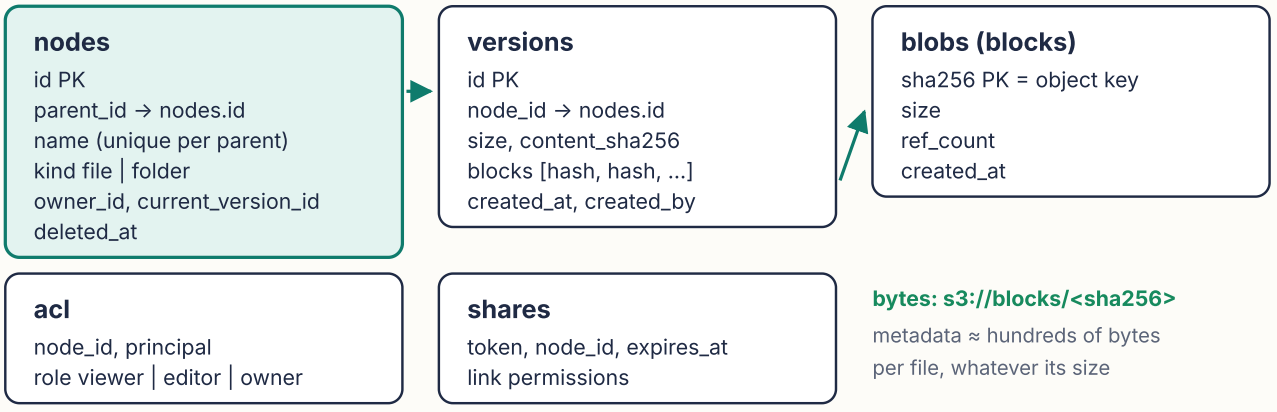


Figure 2. A typical schema: a node tree with parent pointers, immutable versions, content-addressed blobs with reference counts, and ACLs.

The **adjacency list** (`parent_id, name`, unique together) makes rename and move a single-row update, which is what users do most. Resolving a full path walks up the tree, so cache paths or use a recursive query. A **materialized path** column (`/a/b/c`) makes subtree queries trivial but turns moving a folder into rewriting every descendant; a **closure table** answers "all descendants" fast at the cost of extra rows. Permissions are usually stored on folders and inherited, with explicit grants and links (`acl`, `shares`) checked on access.

At scale, shard metadata by owner or namespace (account, shared folder) so a folder listing hits one shard. Dropbox, for example, keeps file metadata in MySQL-backed systems (Edgestore and successors) and the 4 MB blocks in its own block store, Magic Pocket. Metadata grows with the *number* of files, not their size: a billion files at ~500 bytes each is ~0.5 TB before indexes, easily held by a sharded database.

4 Keeping DB and storage consistent

There's no transaction across a database and a bucket, so order the writes so failures leave only *garbage*, never *dangling pointers*:

1. Write the bytes first (immutable, under a new key).
2. Then commit the metadata row that references them, in one database transaction.
3. Garbage-collect objects that no row references, after a grace period (e.g. 24 hours) so an upload that's about to commit isn't collected.

Deletion runs the other way: soft-delete the row (trash), and only when nothing references the content does a background job delete the object.

5 Store a checksum

Store a strong content hash such as **SHA-256** with every file. The client computes it before uploading; the server (or the storage service) computes it on arrival; a mismatch means corruption in transit and the upload is rejected. S3 can verify checksums on upload (`Content-MD5`, or additional algorithms such as CRC32C and SHA-256) and stores them with the object. Later, the stored hash detects silent corruption during migrations, restores or periodic scrubbing. Use SHA-256 rather than MD5 or SHA-1 when the hash also identifies content: practical collisions have been demonstrated for both of those.

6 The hash as the address

In **content-addressed storage** (CAS) the object key *is* the hash: `blocks/9f86d081…`. Identical content produces the same key, so it is stored once and every file row that contains it points to the same object: deduplication for free. Git, Docker/OCI image layers and many backup tools work this way. Bonuses: objects are immutable, so they can be cached forever; uploads are idempotent; integrity is self-checking (re-hash and compare with the key).

Shared content needs **reference counting** (or periodic mark-and-sweep): increment when a version references a blob, decrement when it's deleted, and delete the object only at zero, after the grace period. Before linking a user to an existing blob just because they sent its hash, make sure they actually have the content, or a leaked hash becomes a way to read someone else's file (lesson 8).

7 Versioning

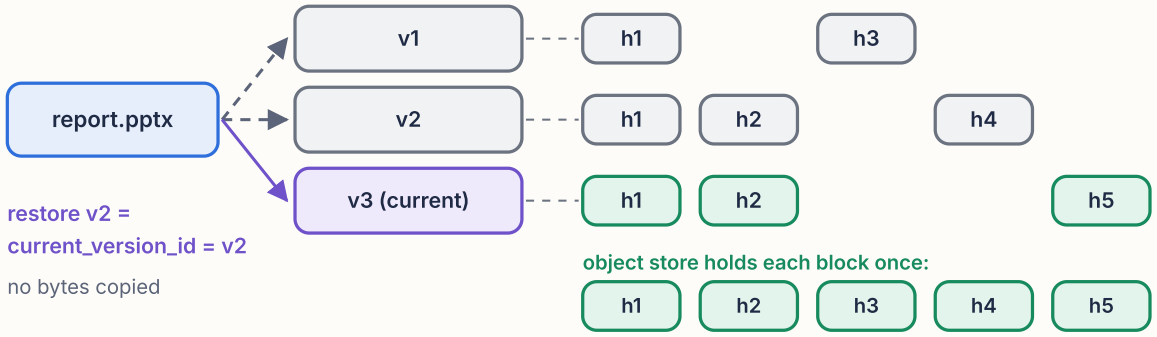


Figure 3. Each version is a row listing its blocks. Unchanged blocks are shared; restoring a version only moves a pointer.

Each save creates a new `versions` row pointing at new content; the file's `current_version_id` moves forward. Old versions keep their pointers, so history costs only the *changed* content when files are stored as blocks. Restore is a pointer update, not a copy. A retention rule ("keep versions for 30 days" or "the last 100") deletes old rows, decrements ref counts, and lets GC free blocks nobody uses.

8 Example: Dropbox-style metadata

WORKED EXAMPLE: TWO USERS UPLOAD THE SAME INSTALLER

A file row holds `id, owner, path, size` and its block list: the SHA-256 of each 4 MB block. A 300 MB installer is 75 blocks. User A uploads it: the client hashes the 75 blocks and asks the server which it lacks; all 75 are new, so it uploads 300 MB, each block stored as `blocks/<sha256>`, then commits one `nodes` row and one `versions` row (ref counts +1). User B saves the same installer: the server already has all 75 hashes, so after an ownership check the "upload" finishes in a second; only a new metadata row is written and ref counts go to 2. B renames it: one row changes. A deletes theirs: ref counts drop to 1 and the blocks stay for B.

TRADE-OFFS

- Two stores mean no shared transaction: you need careful write ordering and garbage collection.
- Content addressing deduplicates and self-verifies, but needs ref counting and complicates per-user deletion and encryption.
- Adjacency lists make moves cheap and path lookups slower; materialized paths are the reverse.
- Block-level versions save space but turn one file into many objects (more requests, more metadata).

COMMON MISTAKES

- Using the user's path as the object key, so renames rewrite bytes.
- Committing metadata before the bytes exist, creating rows that point at nothing.
- Deleting a shared content object when one of its owners deletes their file.
- Trusting a client-supplied hash without verifying it on the server.

9 Key terms

Metadata: facts about a file: name, owner, folder, size, hash, ACL

Checksum: hash used to detect corruption (e.g. SHA-256, CRC32C)

Adjacency list: tree stored as (parent_id, name) rows

Content-addressed storage: objects keyed by the hash of their content

Reference count: number of versions pointing at a blob; delete at zero

Garbage collection: deleting unreferenced objects after a grace period

10 Interview questions

Q1. Where do you store file metadata, and why separately? BASIC

In a database, with the bytes in object storage: metadata is small, structured and changes often (needs indexes, queries, transactions), while content is large and immutable.

Q2. Why store a content hash? BASIC

It catches corruption in transit and at rest, and when the hash is the object key, identical content is stored only once (with a reference count).

Q3. How do you implement 'move folder with 100,000 files'? SCENARIO

With an adjacency-list tree, update the folder's parent_id in one transaction; no object moves. With materialized paths you'd rewrite 100,000 rows, so avoid that model if moves are common.

Q4. The upload succeeded but the metadata insert failed. What's the state, and how do you clean up? SCENARIO

An orphan object with no row; harmless if objects are written first. A GC job deletes unreferenced objects older than a grace period.

Q5. How do you shard the metadata of a billion-file service? SCENARIO

By owner or namespace (account, shared folder), so listings and permission checks stay on one shard; keep a directory from namespace to shard.

Q6. With dedup by hash, how do you safely delete a user's file? TRICKY

Delete their metadata rows and decrement ref counts; only when a blob reaches zero (and after a grace period, to avoid racing with new references) delete the object.

Q7. Why not just use S3 object metadata and tags instead of a database? TRICKY

They can't be queried or indexed, are limited in size, and can't be updated transactionally with other records; you'd need a LIST plus a HEAD per object to answer simple questions.

Next: SD-O-008 · Chunking and Deduplication backendroadmap.com